QUBO Optimization for In-Core Fuel Management

Loading a 37-cell reactor core with simulated annealing

Amr Mohamed · NSF Research Experience for Teachers (RET), University of Houston · Summer 2025
Mentor: Dr. Lei Fan · Program Director: Dr. Weihang Zhu

1. Problem

Between operating cycles, a reactor core is reloaded with a mix of fresh and once-burned ("burnt") fuel assemblies. This project studies a simplified core of 37 cells on a square lattice (Figure 1), in which each cell holds exactly one assembly of one of the two types [1].

Placing two assemblies of the same type next to each other, especially two fresh ones, can create local power peaks and uneven neutron flux [2]. The goal is therefore to place a given fuel inventory so that as few neighbouring cells as possible hold the same type of fuel.

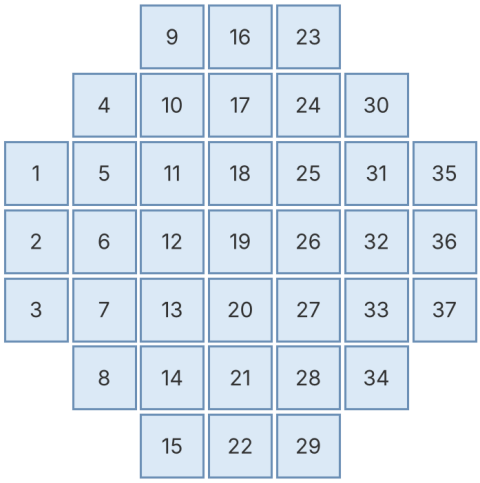


Figure 1. Cell numbering of the 37-cell core. Cells sharing an edge are neighbours (60 pairs in total).

Every layout must satisfy two rules:

- **One assembly per cell:** each cell holds exactly one assembly, either fresh or burnt.
- **Fuel inventory:** the numbers of fresh and burnt assemblies placed must equal the inventory available.

Among all layouts that satisfy these rules, we look for one with the fewest same-type neighbouring pairs.

2. Mathematical Model

Sets and parameters. The core has N = 37 cells, indexed by i. The fuel type is k ∈ {0, 1}, where 0 = burnt and 1 = fresh. E is the set of neighbouring cell pairs (i, j), with |E| = 60. The inventory of type k is N_k, with $N_0 + N_1 = N$.

Decision variables. $x_{k,i} = 1$ if cell i holds fuel of type k, and 0 otherwise. This gives 2 × 37 = 74 binary variables.

Objective. Minimize the number of neighbouring pairs that hold the same fuel type:

$$\min_{x} \sum_{k \in \{0,1\}} \sum_{(i,j) \in E} x_{k,i}\, x_{k,j} \tag{1}$$

Each product equals 1 only when neighbours i and j both hold type k, so the objective counts same-type pairs exactly.

Constraints.

$$\sum_{k \in \{0,1\}} x_{k,i} = 1, \qquad i = 1, \ldots, N \tag{2}$$

$$\sum_{i=1}^{N} x_{k,i} = N_k, \qquad k \in \{0,1\} \tag{3}$$

Constraint (2) places exactly one assembly in every cell, and constraint (3) matches the fuel inventory.

3. QUBO Formulation

A QUBO (quadratic unconstrained binary optimization) problem has a quadratic objective over binary variables and no constraints. It is the input format for simulated annealing samplers and for quantum annealers. Each equality constraint is therefore moved into the objective as a squared penalty with weight λ:

$$H(x) = \sum_{k} \sum_{(i,j) \in E} x_{k,i}\, x_{k,j} \;+\; \lambda \sum_{i} \left(\sum_{k} x_{k,i} - 1 \right)^2 \;+\; \lambda \sum_{k} \left(\sum_{i} x_{k,i} - N_k \right)^2 \tag{4}$$

We used λ = 5. The model was written in JijModeling and converted to QUBO form with its uniform penalty weight [3].

Reading the energy. Every term in H(x) is nonnegative, so H(x) ≥ 0. Any broken constraint adds at least λ = 5. A layout with energy below 5 therefore satisfies every constraint, and its energy equals its number of same-type pairs. An energy of 0 is the lowest possible value.

4. Solution Pipeline

Mathematical model → QUBO (JijModeling) → Simulated annealing (OpenJij) → Decoded layout

The QUBO was solved with OpenJij's simulated annealing sampler (SASampler) using 10,000 reads, and the lowest-energy sample was decoded back into a core layout [4]. Simulated annealing is a classical heuristic that runs on an ordinary computer. The method is quantum-inspired in one sense only: the same QUBO could be sent unchanged to a quantum annealer. All results in this report come from classical runs.

5. Results

We tested two fuel inventories:

Case	Burnt	Fresh	Best energy (simulated annealing)	Same-type pairs in layout	Proven minimum (exact MILP)
1	15	22	3.00	3	3
2	16	21	0.00	0	0

Table 1. Simulated annealing results compared with the proven minimum for each case.

Both energies are below 5, so both layouts satisfy every constraint. In both cases simulated annealing reached the proven optimum (Section 6). Case 2 took about 15 seconds for 10,000 reads.

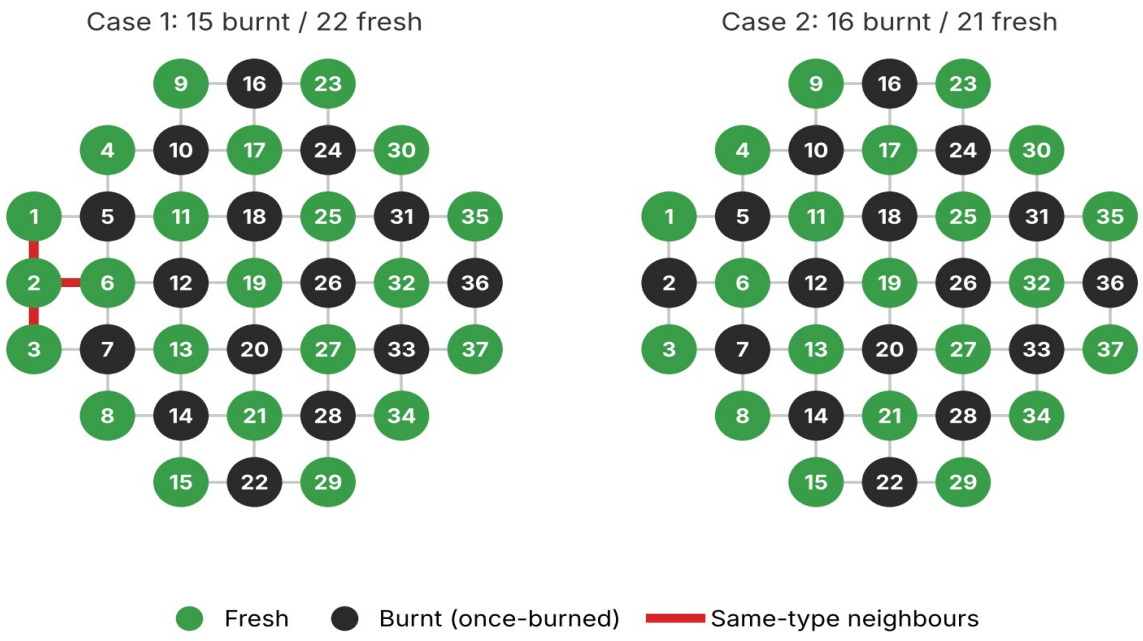


Figure 2. Best layouts found. Red links mark neighbouring cells that hold the same type of fuel.

Case 1 (15 burnt, 22 fresh). A perfect alternation is impossible with this inventory. The best layout has 3 same-type pairs, all between fresh cells near the left edge of the core (cells 1–2, 2–3 and 2–6).

Case 2 (16 burnt, 21 fresh). The best layout has no same-type pairs: it is a checkerboard. Like a chessboard, the lattice splits into two groups of cells, 21 and 16, where every neighbour of a cell lies in the other group. With exactly 16 burnt assemblies, the only perfect layout puts burnt

fuel on the 16-cell group. Because the answer is known in advance, this case serves as a check that the QUBO and the penalty weight are set up correctly.

6. Verifying Optimality

Checking every layout directly would mean examining $C(37, 15) \approx 9.4 \times 10^9$ layouts for Case 1 and $C(37, 16) \approx 1.3 \times 10^{10}$ for Case 2. Instead, the model in Section 2 was solved exactly as a mixed-integer linear program, replacing each product $x \cdot x$ with a standard linearization, using the HiGHS solver in SciPy (script exact_check.py). The proven minima are 3 for Case 1 and 0 for Case 2, matching the simulated annealing results.

At this size the exact solver finishes in well under a second, faster than simulated annealing. The value of this study is therefore not speed: it shows that the QUBO formulation is correct and that simulated annealing finds optimal layouts. Whether it keeps up with exact methods on larger cores is the next question.

7. Limitations and Next Steps

- The model captures only the adjacency rule. It does not compute the core's power distribution or reactivity.
- It uses two fuel types and 37 cells. A typical pressurized-water reactor core holds 193 assemblies from several fuel batches.
- Next steps: add a third fuel type (twice-burned), scale to a 193-assembly core, compare simulated annealing with an exact solver on runtime and solution quality, and study how the penalty weight λ affects feasibility.

Acknowledgements

This work was carried out during the NSF Research Experience for Teachers (RET) program at the University of Houston, under the mentorship of Dr. Lei Fan, with Dr. Weihang Zhu as program director.

References

[1] Galperin, A. (1995). *Nuclear Science and Engineering*, 119, 144–152.

[2] Usmanov, S. R., Salakhov, G. V., Bozhedarov, A. A., Kiktenko, E. O., & Fedorov, A. K. (2023). *Quantum and quantum-inspired optimization for an in-core fuel management problem*. arXiv:2308.13348 [quant-ph].

[3] Jij Inc. JijModeling: a mathematical modeling tool for optimization problems. https://www.jijzept.com

[4] Jij Inc. (2025). OpenJij: a framework for Ising and QUBO optimization (Version 0.10.7). https://www.openjij.org

Code

Notebook (In_core.ipynb) and exact check (exact_check.py): github.com/MAmr9UH/Nuclear-Reactor-Core-Optimization-Quantum-Computing